

LLM-based Personalized Travel Planner

First Author¹ and Second Author¹

Affiliation / Address line 1, Affiliation / Address line 2,
Affiliation / Address line 3, email@domain

Abstract. Effective travel planning requires handling multiple information sources, managing constraints, and tailoring recommendations to user preferences. While LLMs have introduced dynamic, context-aware decision-making, single-agent models often struggle with complex multi-step reasoning, constraint adherence, and modular execution. To overcome these limitations, we propose a hierarchical multi-agent framework where a supervising chatbot agent coordinates specialised agents for schedule validation, user query formulation, real-time data retrieval, and constraint-aware itinerary generation. By integrating LLM reasoning, external APIs, and iterative refinement mechanisms, our approach ensures adaptability, efficiency, and personalization. Evaluations on real-world datasets demonstrate superior scalability, modularity, and constraint satisfaction, positioning our system as a robust solution for AI-driven travel planning.

Keywords: multi-agent systems, personalized recommendations, constraint optimization, hierarchical task planning, automated travel systems

1 Introduction

Travel planning is a complex and time-consuming task, requiring users to navigate multiple sources such as calendars, booking platforms, and travel guides to organize a trip. Traditional rule-based travel systems, dependent on static databases, offered limited personalization and adaptability [2]. The advent of Large Language Models (LLMs) has transformed automated travel planning by enabling dynamic, context-aware recommendations tailored to user preferences [1]. Unlike traditional systems, LLMs handle unstructured data and adapt to evolving constraints. However, early LLM-based travel planners struggled with multi-step reasoning, constraints management, and task adherence. To address these challenges, advanced prompting techniques such as Chain of Thought (CoT) [24] and Tree of Thought (ToT) [30] have been introduced to enhance structured reasoning.

Additionally, frameworks such as ReAct [31] enabled LLMs to interact with external APIs, allowing real-time data retrieval and better decision-making. Despite these advancements, single-agent LLMs struggle with complex, multi-faceted planning tasks [21]. Building on this, multi-agent systems have been developed, where specialized agents collaborate to improve modularity, scalability,

and efficiency [19, 26]. Additionally, new benchmarks for evaluating LLM-based planning have provided tools for assessing and refining these models [29, 33].

In this paper, we propose a hierarchical multi-agent framework for personalized travel planning, where a supervising chatbot agent coordinates multiple specialized agents to streamline the planning process. Our approach integrates LLM-based reasoning, external APIs, and multi-agent collaboration to generate personalized and adaptive itineraries. We evaluate its effectiveness using real-world datasets and performance metrics, demonstrating its advantages over traditional approaches.

2 Related Work

2.1 Enhancing LLM Personalization with Long-Term Memory

Large Language Models (LLMs) have revolutionized natural language processing, demonstrating advanced text comprehension and generation capabilities [1, 22]. The pre-training of LLMs on massive text and code datasets equips these models with a broad understanding of language, grammar, and reasoning abilities [1]. Despite their potential, LLMs pose challenges related to complex reasoning and problem-solving. Recent work, including contributions from DeepSeek [6] and OpenAI [35], has focused on improving these abilities and moving toward more generalized intelligence [8].

One critical limitation of LLMs is their lack of long-term memory, which affects personalization and contextual continuity in applications like travel planning. Integrating memory mechanisms can improve personalized recommendations and sustained user engagement [12]. For instance, Memory Bank [36] functions as a structured repository that retains past conversations, event summaries, and user personality assessments to enhance long-term user understanding.

Addressing this limitation is crucial for developing LLM-driven personalized systems, where user preferences and past interactions inform responses.

2.2 Integrating LLMs with Real-time Data

Integrating LLMs with real-time data sources via APIs is essential for creating dynamic, contextually relevant travel planning applications that provide accurate, up-to-date recommendations, thereby enhancing the user experience [16]. Online planning mechanisms further ensure itineraries adjust dynamically based on API feedback [18]. To achieve this, frameworks that enable LLMs to interact seamlessly with APIs are essential. RestGPT is one example that connects LLMs with RESTful APIs using a coarse-to-fine online planning mechanism for task decomposition and API selection [18]. TaskMatrix.AI [13] extends this concept, envisioning a broader AI ecosystem where large language models act as a central "brain" while external APIs serve as specialized task solvers. Work on utilizing LangChain highlights how streamlining LLM applications with API integration enhances real-world functionality [20].

2.3 Building User Profiles and Personalized Experiences

Accurate and personalized travel recommendations require a deep understanding of individual user preferences, interests, and constraints. Classical user-profiling surveys summarize feature engineering and behavior-based models for recommender systems [10], but they do not address how profiles should be represented, updated, and queried inside tool-augmented LLM applications that must reason over evolving conversational context. Work on leveraging translation models to improve recall of user-specific information demonstrates gains for retrieval and personalization [15], yet it incurs additional compute and leaves open how such signals are fused with planning and constraint satisfaction in end-to-end agents.

Recent advancements in persona-based profiling offer a structured approach to user modeling. Building on persona generation research [4, 32], new methodologies create diverse persona datasets for conversational AI, grounded in structured user attributes such as occupation, interests, and educational background.

2.4 LLM-Powered Task Planning

The application of LLMs in itinerary generation has gained significant attention, with various approaches demonstrating unique advantages and trade-offs. LLM-centric methods leverage structured prompts and multi-agent collaboration to enable multi-phase planning and simulate human-like reasoning [28]. However, standalone LLMs face inherent limitations in handling complex planning tasks. To address this, recent developments incorporate external tools and structured environments to enhance the decision-making process [9, 11]. The AGENTGEN framework exemplifies this hybrid approach by automating task generation in various environments, improving LLM-driven planning [9]. Other work integrates Satisfiability Modulo Theories (SMT) solvers to ensure strict adherence to constraints in itinerary generation [7].

TRIP-PAL [16] introduces a constraint-aware travel planning framework that integrates symbolic planning with LLMs. While it demonstrates strong performance on its own benchmarks, the specification and code are not publicly released, and its evaluation setup differs from ours. We therefore do not include it in our experiments but discuss it as complementary prior work.

2.5 Multi-Agent Systems and Collaboration

An agent is an autonomous or semi-autonomous entity, typically an LLM or an LLM augmented with tools, assigned to carry out a specific task within a system. Tools are external resources such as APIs, databases, or executable functions that agents use to extend their capabilities, access real-time information, and interact with external systems beyond the LLM’s internal knowledge.

Multi-agent systems leverage these agents and tools to enhance LLM-based applications by breaking down complex tasks into manageable sub-tasks and enabling efficient collaboration among agents [19, 26]. Their modularity and

flexibility allow specialized agents to handle different aspects of travel planning, such as transportation, accommodation, and activity recommendations [26].

Recent frameworks have operationalized these concepts for real-world applications. AutoGen implements conversable agents and conversation programming to enable dynamic agent interactions, supporting both static and adaptive conversation patterns [26]. The human-like planning framework [28] applies a multi-agent system to travel planning, assigning agents to tasks such as outline generation, information retrieval, and itinerary formulation. Unlike AutoGen’s general-purpose design, this framework emphasizes multi-phase planning and sequential decision-making.

Additionally, emerging frameworks like LangChain and LangGraph provide structured environments for building LLM-driven agents and workflows. LangChain enables tool integration, memory management, and agent routing, while LangGraph extends this with graph-based execution logic for orchestrating complex, stateful multi-agent interactions. These frameworks help streamline the development of adaptive, tool-augmented LLM systems across domains, including travel planning.

3 Methodology

Our multi-agent system follows a hierarchical structure shown in Fig. 1 where a supervising chatbot agent serves as the central coordinator, controlling and orchestrating interactions between specialized agents while maintaining direct communication with users.

The system is implemented as a state graph consisting of nodes (specialized agents) and edges (transition conditions) [11, 23], where each agent handles specific aspects of travel planning such as calendar management, data retrieval, itinerary generation, and user feedback processing. After each agent completes its task, it notifies the supervising chatbot of its output and updated state information.

The chatbot then determines the next course of action based on the current context and user requirements. This decision-making process can either route the workflow to another specialized agent for further processing or request additional input from the user to clarify requirements or gather missing information.

This hierarchical approach enables the system to handle complex, multi-step travel planning tasks while maintaining conversation context and providing personalized recommendations. The state graph representation allows for dynamic routing and ensures that each user interaction builds upon previous context, creating a seamless and intelligent travel planning experience.

3.1 Agents and Modules

Chatbot Agent (Supervisor) The chatbot agent serves as the central orchestrator of the hierarchical multi-agent system [3], ensuring seamless coordination among specialized agents and managing the overall flow of the travel planning

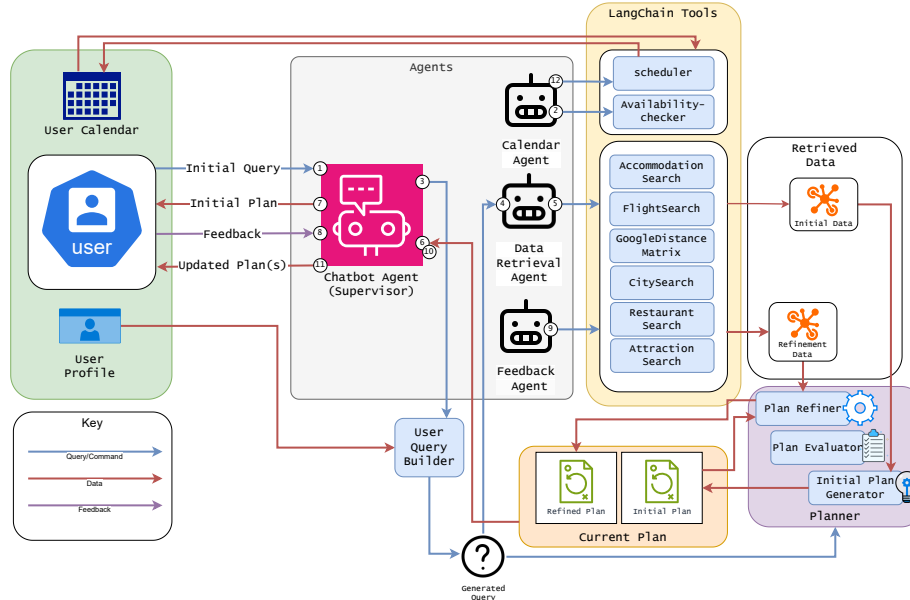


Fig. 1. Multi-Agent System Architecture: All other agents are controlled by the *Chatbot Agent*, but we have opted not to draw the control and communication lines between the agents to reduce unnecessary clutter. When a command or a data item is relevant to all the entities in a parent entity, the relevant arrow terminates on the parent entity. Otherwise, it terminates on the specific relevant child entity. The numbers on the agents at arrow terminals indicate the order in which each action may happen in a typical execution. Actions 6 and 10 are functionally identical but happen at different points in execution due to the existence of the feedback loop. All steps in the feedback loop (8-11) may repeat indefinitely until the user is satisfied.

process. It directs tasks dynamically to the appropriate modules based on context, user requirements, and system state, while maintaining coherent dialogue as the primary interface with users.

To preserve context, the chatbot manages JSON-based state storage, enabling recovery after restarts and ensuring continuity across multiple sessions. At critical stages, it integrates human-in-the-loop decision points through the **human.interrupt** node, allowing users to confirm, adjust, or extend inputs. Additionally, it supports a real-time streaming architecture that delivers updates as plans are generated while maintaining state consistency.

Thus, the chatbot combines workflow orchestration (see Section E.2), conversation management, and decision-making into a unified supervisory role.

Calendar Agent The calendar agent is responsible for managing the user’s schedule using the Google Calendar API [17]. It is an agent with two tools: a **calendar availability-checking tool** and an **event-scheduling tool**. Calen-

dar agent uses the calendar availability-checking tool to verify whether the user is available on the selected travel dates. If the given dates are unavailable, the chatbot will prompt the user to provide alternative dates. Once a travel plan is confirmed and finalized, the agent uses the event-scheduling tool proposed by Wijerathne et al. [25] to add the trip details to the user’s calendar.

User Query Builder Module The User Query Builder Module constructs comprehensive travel queries by combining user-provided details (such as origin, destination, and travel dates) with stored profile preferences. When preferences like food, travel style, or dietary restrictions are missing, the module supplements them automatically from the user profile. It validates and structures this information into a standardized format, ensuring completeness and accuracy before forwarding the query to the Data Retrieval Agent and Planner Module. Additionally, it identifies gaps in essential information and prompts the user for clarification when required. This preprocessing step ensures that all relevant details are properly captured and formatted for subsequent planning stages.

Table 1. Overview of data retrieval tools and their functions.

Tool	Function
CitySearch	Search cities within a specified region.
FlightSearch	Search flights for a specific date between two cities.
DistanceMatrix	Search driving distance, time, and cost between cities.
RestaurantSearch	Search restaurants in the given city.
AttractionSearch	Search attractions in the given city.
AccommodationSearch	Search accommodations in the given city.

Data Retrieval Agent The Data Retrieval Agent is responsible for gathering comprehensive travel information from multiple heterogeneous sources. It integrates six specialized LangChain tools [29], summarized in Table 1, each designed to query a distinct dataset or API and return structured results. Collectively, these tools cover flights, accommodations, restaurants, attractions, and transportation, ensuring complete coverage of user travel needs.

Data Sources and Integration: The agent operates on diverse datasets, including flight schedules and prices, restaurant data, accommodation listings, tourist attractions, city-state mappings, and the Google Distance Matrix for travel times and costs. All datasets undergo preprocessing, including null value removal, column filtering, and city name normalization. This enables robust querying and consistent results across multiple data types.

Tool Implementation: Each tool is implemented as a LangChain-compatible component [20] using the `@tool` decorator, which allows the supervising ReAct

agent to call them dynamically during execution. The system is designed so that one tool can handle flight information by retrieving routes, times, and prices, while another can process accommodation details such as availability, pricing, and policies. Validation checks are built in to ensure that user inputs, such as dates or city-state combinations, are correct. When invalid inputs are detected or requested information is unavailable, the agent provides clear and informative error messages to guide the user.

Intelligent Data Processing: The agent performs multiple preprocessing steps, including string normalization, data type conversions, and city-state validation. It incorporates advanced calculations such as distance-based transportation cost estimation and comparative analysis of modes (e.g., flight vs. taxi). Data integrity is preserved through structured intermediate storage and aggregation.

ReAct-Based Retrieval with Chain of Thought: The Data Retrieval Agent employs the Reasoning and Acting (ReAct) [31] paradigm, which integrates Chain of Thought (CoT)[24] reasoning into its retrieval process. The agent reasons step by step about the information required, selects the appropriate tool, observes the output, and refines its queries until all relevant data are collected. This approach enables the agent to adjust its strategy in response to partial or missing results and to recover gracefully from tool failures. For example, when planning a trip, it may first retrieve flight options, then proceed to accommodations, and finally expand the search to restaurants and attractions, with each step guided by the outcomes of the previous one. By combining sequential reasoning with dynamic tool invocation, the agent maintains logical consistency, adapts to evolving queries, and ensures the delivery of complete, constraint-aligned information for itinerary generation.

State Management and Streaming: Retrieved information is stored in temporary memory structures (e.g., `notebook.data`) [29] and then committed to the global system state for use by downstream modules such as the Planner and Feedback Agents. Results are tagged by type (flight, hotel, restaurant, etc.) for structured organization. A streaming architecture, implemented via LangGraph’s `create_react_agent`, ensures that partial results are communicated in real time, allowing the supervising chatbot to provide progressive updates to the user.

Error Handling and Robustness: The agent incorporates robust error management strategies, including validation checks, fallback responses, and graceful degradation when partial data are missing. Instead of failing silently, the system informs the user of unavailable data while still delivering accessible results from other sources.

Planner Module The Planner Module represents the core intelligence engine of the TravelPlanner system, responsible for generating, evaluating, and iteratively refining travel itineraries until optimal quality thresholds are achieved. It is

organized into a three-component architecture comprising the *Initial Plan Generator*, the *Plan Evaluator*, and the *Plan Refiner*. Together, these components transform basic user requests into optimized and constraint-compliant travel plans through a structured generate–evaluate–refine cycle.

The Initial Plan Generator serves as the foundation of the planning process. It constructs baseline itineraries by combining structured templates with intelligent constraint satisfaction mechanisms. Templates define a standard itinerary structure and ensure completeness across categories such as transportation, meals, attractions, and accommodation. Within this framework, the system employs a dynamic budget allocation engine that replaces traditional fixed ratios with a context-aware distribution. Budget shares are adapted to trip type, distance, duration, destination cost index, and, when available, user preferences. For example, business trips prioritize accommodation and transportation, while day trips exclude accommodation and allocate more resources to activities and dining. Long trips increase the proportion allocated to accommodation, whereas short trips reduce it. Expensive destinations increase spending on lodging and dining, while cheaper destinations allocate relatively more to activities. This allocation process incorporates user-defined preferences through convex blending, ensuring personalization while maintaining feasibility. A constraint satisfaction engine validates compliance with user requirements such as budget limits, trip duration, and mandatory locations, and assigns confidence scores that penalize violations. To enrich generated content, large language models are integrated into the process, and fallback templates guarantee robustness when advanced generation fails.

Once a draft itinerary is produced, the Plan Evaluator applies a deterministic, multi-dimensional scoring framework to assess its quality. Plans are evaluated across five weighted dimensions: budget compliance (30%), time efficiency (25%), geographic logic (20%), activity diversity (15%), and constraint satisfaction (10%). Each dimension employs graduated scoring scales. For instance, budget compliance awards 100 points when under budget by 10% or more, 90 points for exact compliance, and progressively fewer points for increasing overruns until a score of zero for plans exceeding the budget by more than 30%. Time efficiency is measured relative to an optimal allocation of eight hours of activities and two hours of travel per day, while geographic logic evaluates total travel distance against an optimal route to penalize unnecessary backtracking. Activity diversity measures the distribution across cultural, recreational, dining, shopping, and sightseeing categories, and constraint satisfaction applies penalties for hard and soft violations. A plan is deemed acceptable only if its overall score reaches at least 75/100, with each individual dimension scoring at least 60/100. Pseudocode for this evaluation process is shown in Fig. 2. In addition to quantitative scores, the evaluator produces qualitative recommendations that identify weaknesses and suggest targeted improvements.

The Plan Refiner implements iterative improvement strategies to address deficiencies identified by the evaluator. It applies sequential strategies guided by dimension scores, focusing first on areas that score below 70 points. Budget

Rule-Based Evaluator (pseudocode)

```

# Input: itinerary text, user_budget, constraints
# Output: overall score S, dimension scores, accept/refine flag

weights = {budget:0.30, time:0.25, geo:0.20, diversity:0.15,
           constraints:0.10}

def evaluate(plan, budget, constraints):
    s_budget = score_budget(plan, budget) # 100 if <=90% budget, 0
    if >130%
    s_time = score_time(plan) # target: 8h activity + 2h travel/day
    s_geo = score_geo(plan) # ratio of route vs. 2-opt heuristic
    s_div = score_diversity(plan) # entropy over {culture, rec,
    dining,...}
    s_const = score_constraints(plan, constraints) # -50 hard, -20
    soft violations

    scores = {budget:s_budget, time:s_time, geo:s_geo,
             diversity:s_div, constraints:s_const}

    S = sum(scores[d] * weights[d] for d in scores)

    accept = (S >= 75 and min(scores.values()) >= 60)
    return S, scores, accept

```

Fig. 2. Deterministic rule-based evaluator used to score itineraries across five dimensions.

optimization is achieved by substituting costly elements with more economical alternatives, such as shifting from luxury to mid-range accommodations or from fine dining to casual options. Time optimization reorganizes schedules to improve temporal flow, groups nearby activities to minimize travel time, and ensures logical sequencing throughout the day. Constraint violations are corrected through targeted modifications that address budget, time, and location requirements without compromising overall quality. Diversity enhancement mechanisms enrich the itinerary by adding underrepresented activity categories, while geographic optimization reduces travel distances through efficient routing. These refinements are supported by language model prompts that maintain itinerary structure while implementing improvements. Each refined plan is re-evaluated, and the process repeats until quality thresholds are met.

The optimization loop continues until termination conditions are satisfied. These include reaching or exceeding the minimum quality threshold, exceeding the maximum number of five iterations, failing to achieve a minimum improvement of five points between iterations, or surpassing the global timeout of 300

seconds. This ensures that optimization remains efficient, prevents infinite loops, and guarantees responsiveness. Throughout this process, the system maintains detailed records of evaluation results, refinement metadata, and optimization history to enable performance monitoring and continuous improvement. The module is designed with robustness in mind, featuring fallback mechanisms that degrade gracefully when advanced strategies fail and comprehensive error handling to ensure stability.

Throughout this process, the system tracks evaluation results, applied refinements, and optimization history, enabling performance monitoring and continuous improvement. The Planner Module’s contributions include its dynamic budget allocation, systematic generate–evaluate–refine cycle, and multi-dimensional scoring methodology, which together establish a novel and effective approach to automated travel planning.

Feedback Agent The Feedback Agent extends the TravelPlanner system by providing selective modification of generated itineraries in response to user requests. Rather than regenerating entire plans, the agent isolates the scope of a requested change and invokes only the necessary planning components. This selective invocation framework significantly reduces computational overhead and preserves unaffected elements of the itinerary, ensuring continuity across revisions. Change detection is achieved through natural language parsing of user feedback, which enables classification of modifications into temporal, spatial, financial, and preference-based categories. Each category is mapped to the corresponding planning submodules, with dependency management ensuring that prerequisite adjustments are applied in the correct sequence.

To maintain integrity during the update process, the agent employs a state management system with full version control, allowing rollback to previous configurations if modifications fail. All adjustments are validated against budgetary, temporal, and geographic constraints, with automated conflict resolution applied where possible and user notification triggered when resolution requires intervention. Modifications that alter core itinerary parameters, such as budget or trip duration, propagate through dependent elements via cascading update logic, ensuring global consistency across the travel plan.

Quality assurance is embedded directly into the feedback cycle. Following each adjustment, the Plan Evaluator is re-engaged to assess compliance with multidimensional scoring thresholds, and results are compared against the baseline itinerary to confirm that plan quality is preserved or improved. Updated plans are communicated to the supervising chatbot agent alongside structured summaries that outline the modifications and their impacts.

4 Experiments

4.1 Experiment Setup

We evaluate two strategies from the single-agent TravelPlanner benchmark [29]: **Sole Planning** (all information provided upfront) and **Two-Stage Planning** (retrieval before planning); details are in Appendix A.

Baselines: We include two single-agent baselines with GPT-4o (gpt-4o-2024-05-13): CoT (no external tools) and ReAct (action/observation loops without multi-agent orchestration). Consistent with TravelPlanner [29], these baselines struggle on constraint-heavy itineraries; we re-ran them in our setup for a fair comparison. We exclude TRIP-PAL [16] due to unavailable code and mismatched evaluation protocols.

Metrics: Following the benchmark, we report *Delivery Rate*, *Commonsense Constraint Pass Rate*, *Hard Constraint Pass Rate*, and *Final Pass Rate*, with both micro- and macro-averaging. Formal definitions and a worked example are given in Appendix B.

Because TravelPlanner is single-agent while our framework is hierarchical and multi-agent, a direct head-to-head comparison is inappropriate. Therefore, Sections 4.3 and 4.4 report single-agent experiments to isolate the effect of our refinement mechanisms and ensure fair comparison to prior work.

4.2 Datasets

We conduct all experiments on the **TravelPlanner** benchmark [29], a closed-world travel-planning environment with a tool-augmented sandbox designed to minimize data contamination and to test constraint satisfaction under realistic retrieval and planning workflows. The benchmark contains **1,225 queries** covering multiple trip lengths and difficulty levels, and is split as follows: **train:** 45 query-plan pairs (5 queries per group), **validation:** 180 queries (20 per group), and **test:** 1,000 queries. Each query requires an itinerary that includes transportation, daily meals, attractions, and accommodation, while satisfying both commonsense and hard constraints in the environment.

Table 2. TravelPlanner summary used in our experiments.

Total queries	1,225 (train 45, val 180, test 1,000)
Trip lengths	3, 5, 7 days
Difficulty	easy / medium / hard
Geographic spread	≥ 192 origins, ≥ 127 destinations (test split)
Budgets (ours, for analysis)	<\$1K, \$1K–\$3K, >\$3K buckets
Environment	Closed-world sandbox with offline tools (no live APIs)
Licensing	CC BY 4.0 (dataset), MIT (code)

Model Configuration: All experiments used GPT-4o (gpt-4o-2024-05-13). The generation parameters were tuned to balance creativity and reproducibility. Specifically, we used:

- Temperature: 0.7 for creative planning, 0.3 for evaluation
- Max tokens: 4000 (planning), 1000 (evaluation)
- Top-p: 0.9 (planning), 0.8 (evaluation)
- Frequency penalty: 0.1
- Presence penalty: 0.1

For refinement, we allowed up to 5 iterations with a global timeout of 300 seconds.

4.3 Budget Constraint Optimization

One of the critical factors affecting the final pass rate of generated plans is the budget constraint, a hard constraint that directly determines itinerary feasibility. To mitigate failures from budget violations, we implemented an automated budget refinement mechanism. The process is iterative: an initial itinerary is generated and checked against the budget. If it fails, adjustments are applied up to three times; if it still fails, the plan is marked unsuccessful. This refinement is necessary because LLMs often exceed budgets due to limited cost-awareness during generation. Without refinement, such violations would yield high rejection rates. By contrast, iterative refinement significantly reduces budget-related failures and improves the final pass rate.

Averaging conventions. *Micro* pass rate treats each constraint equally across all plans (total constraints satisfied divided by total applicable constraints over the suite). *Macro* pass rate is stricter and counts a plan only if *all* of its applicable constraints are satisfied, then averages this indicator over plans. *Final Pass* requires: (i) a schema-valid itinerary is delivered, (ii) all commonsense constraints pass, and (iii) all hard constraints pass.

Table 3. Budget Refinement Performance Analysis. “Unrefined” corresponds to the original single-agent TRAVELPLANNER baseline (no refinement). “Refined” applies our budget refinement loop, with all other settings identical.

Metric	Unrefined (Baseline)	Refined
Delivery Rate	100%	100%
Commonsense Constraint Micro Pass Rate	88.875%	81.125%
Commonsense Constraint Macro Pass Rate	35%	35%
Hard Constraint Macro Pass Rate	50%	60%
Final Pass Rate	25%	35%

4.4 Constraint Set Optimization

Beyond budget constraints, we extend our refinement approach to address the entire set of constraints imposed on itinerary generation. Large Language Models in zero-shot prompting often struggle to produce fully optimized results on the first attempt due to misunderstandings and ambiguities in constraint interpretation. To counter this, we introduce a constraint refinement mechanism based on plan scoring. Each generated plan is assigned a score based on the number of satisfied constraints. If the score meets a predefined threshold, the plan is accepted. If it fails, iterative refinement is applied: the refined plan is compared against the previous version, and if its score improves, it replaces the prior iteration. The process continues until either the threshold is met or the maximum limit is reached.

Table 4. Plan Refinement Performance Analysis. “Unrefined” corresponds to the original single-agent TRAVELPLANNER baseline (no refinement). “Refined” applies our constraint refinement loop, with all other settings identical.

Metric	Unrefined (Baseline)	Refined
Delivery Rate	100%	100%
Commonsense Constraint Micro Pass Rate	88.875%	91.875%
Commonsense Constraint Macro Pass Rate	35%	55%
Hard Constraint Macro Pass Rate	50%	60%
Final Pass Rate	20%	45%

4.5 Personalization evaluation

In the travel plan generation process, the planner constructs an itinerary by considering the user query and generating a plan aligned with stated preferences. However, not every query explicitly mentions user preferences. To address this, a user profile is utilized in addition to the user query to enhance itinerary personalization. This necessitates a structured evaluation procedure to assess how well the generated plans align with user preferences.

While human evaluators can assess itinerary personalization, the process is time-consuming and subject to bias or inconsistency. Human judges may apply varying scoring criteria, leading to variations in ratings. In contrast, LLMs follow predefined evaluation criteria, ensuring uniformity and scalability. Therefore, we employ GPT-4o as a role-playing LLM to evaluate itinerary personalization, following the LLM-as-a-Judge approach [14, 27, 34], which provides quantitative measures of response alignment with user profiles. Prior work [5, 14] have also demonstrated the effectiveness of using GPT-4o in role-playing evaluation settings, further validating our choice. To assess reliability, we conducted an experiment comparing LLM evaluations with human judgments. A dataset of 100 travel plans was used, each designed with varying levels of personalization. Three

human judges independently rated the itineraries based on user profile and user query alignment, using a 1-to-5 rating scale defined in Appendix C. GPT-4o (LLM) was prompted to evaluate the same itineraries without roleplaying as a traveler, strictly acting as a neutral judge. The primary objective was to measure the alignment between LLM ratings and human ratings. The results show a significant improvement in personalization quality when incorporating user profiles. Plans generated using only the user query achieved an average personalization score of 40%, whereas plans generated using both the user query and user profile achieved 87%, as assessed by GPT-4o. This highlights the critical role of structured user modeling in producing personalized travel experiences.

To quantify rating consistency between evaluators, we computed Cohen’s Kappa coefficient, a standard metric for agreement measurement. Additionally, we applied linear-weighted Cohen’s Kappa, which penalizes minor disagreements less severely (e.g., a rating of 3 vs. 4 is weighted lower than a disagreement between 1 vs. 5). This provides a more realistic measure of consistency in scoring. The average weighted Cohen’s Kappa reaches 0.8151, indicating strong agreement between human and LLM judges and validating the reliability of our auto-evaluation method.

4.6 Ablation Study

To measure the effect of individual components in our framework, we conduct an ablation study. Each system variant is tested under identical datasets, prompts, and evaluation protocols to ensure fairness. Evaluation follows the metrics in Section B, using GPT-4o as judge with consistent prompt templates.

Protocol: We evaluate four variants of our framework: (i) *Full System* (Supervisor + Planner + Evaluator + Feedback Agent + user profile), (ii) *No Evaluator*, (iii) *No Feedback Agent*, and (iv) *No Eval & Feedback* (Planner only). In addition, we re-implemented the *TravelPlanner (Two-Stage)* baseline [29] under our experimental setup and report its results for comparison. All receive identical retrieval results and user queries where applicable.

Table 5. Component Contribution Analysis Across All Metrics. Scores are means over $K=5$ runs.

Condition	Personalization	Final Pass	Hard Constraint	Commonsense	Delivery
	Score (%)	Rate (%)	Pass Rate (%)	Pass Rate (%)	Rate (%)
Full System	87	52	68	62	100
No Evaluator	79	30	45	49	91
No Feedback Agent	71	28	42	46	90
No Eval & Feedback	65	21	35	40	86
TravelPlanner (Two-Stage; ours re-run)	56	10	28	15	89

Component Contribution. Table 5 shows that the *Full System* outperforms all reduced variants. Removing either the Evaluator or Feedback Agent nearly halves

the *Final Pass Rate* (52% $\rightarrow$ 28–30%) and lowers *Hard Constraint Pass Rate* by over 20 points, indicating their complementary roles: the *Evaluator* flags constraint violations, while the *Feedback Agent* applies corrections. Removing both causes the steepest decline (Final Pass Rate 21%). Personalization and commonsense scores also decline but less sharply, while delivery remains relatively high across all settings, suggesting surface fluency is easier to maintain than deeper constraint satisfaction. These results underline the importance of modular design, especially the *Evaluator* and *Feedback Agent*, for achieving robust task success.

5 Discussion

This work demonstrates that combining explicit control (Supervisor routing), deterministic critique (rule-based Evaluator), and selective editing (Refiner) yields reliable gains in constraint satisfaction and end-to-end success for travel planning. Rather than relying on implicit “memory” or emergent behavior, the system enforces clear decision points and measurable signals.

The generate $\rightarrow$ evaluate $\rightarrow$ refine loop provides a transparent mechanism: the Supervisor enforces stage boundaries, the Evaluator applies a weighted scoring rule with fixed thresholds (Fig. 2), and the Refiner applies targeted changes guided by those scores. Empirically, ablations (Table 5) show that removing either the Evaluator or the Refiner nearly halves the Final Pass Rate, confirming their complementarity. Although selective refinement adds moderate overhead (30–300 s per plan, see Appendix D), the increased success on hard constraints offsets these costs. Overall, the results support our claim that deterministic evaluation paired with selective refinement can improve reliability in multi-agent planning.

6 Future Work

6.1 Multi-Modal Agent Communication

While our hierarchical architecture enables high-level coordination, current inter-agent communication remains limited to state passing and basic routing, creating bottlenecks as task complexity grows. Future work will introduce semantic message exchanges that include intent, assumptions, evidence, and confidence levels, allowing agents to interpret context more effectively. A shared queryable memory will support knowledge transfer with provenance tracking and controlled access. We also plan to formalize protocols for planning, critique, and learning, and to implement message prioritization based on urgency and relevance. These advances aim to improve coordination, reduce redundancy, and increase robustness in itinerary generation and revision.

6.2 Advanced Learning Integration

Currently, learning components operate in isolation and cannot fully leverage distributed knowledge. We plan to enable cross-agent learning so that heuristics and insights gained by one agent can inform others. Local observations will be aggregated into global summaries accessible across the system, with update rules adapted to agent roles and interaction contexts to avoid negative transfer. Learning will propagate through structured push-pull mechanisms with conflict resolution and freshness guarantees. This integration is expected to improve personalization, constraint satisfaction, and efficiency, reducing the need for repeated refinements.

6.3 Distributed Supervisor and System Scalability

The current centralized supervisor is effective for single users but becomes a bottleneck under concurrent workloads. To address this, we propose a distributed supervisor design that maintains conversational coherence while enabling horizontal scalability and fault tolerance. Multiple supervisor instances will operate in parallel, coordinated by a load balancer that enforces session affinity and re-assigns sessions on failure. Session state will be externalized to a Redis-backed store for stateless execution and seamless failover. A dynamic agent registry, fully asynchronous backend, and containerized deployment with reverse proxy support will provide efficient load distribution and resilience. Real-time monitoring of active sessions and agent utilization will further support autoscaling. This approach is intended to deliver scalable, fault-tolerant performance suitable for real-world deployments.

7 Conclusion

We presented a multi-agent planning framework that combines explicit Supervisor routing, a deterministic rule-based Evaluator, and selective refinement to generate reliable, constraint-aware travel itineraries. Through a structured generate-evaluate-refine cycle, the system achieves higher success rates on hard constraints and commonsense checks than single-pass prompting, while maintaining modest computational cost.

Our results suggest that deterministic evaluation paired with targeted edits provides a reproducible and auditable pathway to improve LLM-based planning. While current performance is limited by model capabilities, dataset scope, and the absence of human-centered validation, these constraints point toward natural extensions. Future work will explore integration with live data sources, richer personalization signals, and user studies to assess satisfaction. Beyond travel planning, the same pattern of evaluation and refinement can generalize to other structured decision-making domains such as event scheduling or meal planning.

8 Limitations

This study has several limitations. First, the planning quality remains bounded by the capabilities of current large language models, which cannot yet match the nuanced judgment of experienced human planners. Second, the iterative refinement loop improves reliability but introduces additional latency (30–300 s per plan), which may affect responsiveness in interactive use. Third, evaluation was conducted on a U.S.-based dataset, limiting generalizability to other cultural and geographic contexts; expanding to global travel data would strengthen external validity. Fourth, personalization is based on limited user inputs and does not yet incorporate richer signals (e.g., from personal calendars or communication data), though doing so would raise important privacy and security considerations. Finally, our evaluation relied on rubric-based scores rather than human-centered studies, so alignment with user satisfaction remains to be established.

References

1. Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al.: Language models are few-shot learners. *Advances in neural information processing systems* **33**, 1877–1901 (2020)
2. Chen, P.: Retracted: Design of travel itinerary planning system based on artificial intelligence. *Journal of Physics: Conference Series* **1533**(3), 032078 (2020)
3. Duan, Z., Wang, J.: Exploration of llm multi-agent application implementation based on langgraph+crewai (2024), URL <https://arxiv.org/abs/2411.18241>
4. Ge, T., Chan, X., Wang, X., Yu, D., Mi, H., Yu, D.: Scaling synthetic data creation with 1,000,000,000 personas. *arXiv preprint arXiv:2406.20094* (2024)
5. Gu, S.: Intelligent applications and practical research of gpt-4o in self-guided tour route planning. *Asian Journal of Advanced Research and Reports* **19**(3), 323–336 (2025)
6. Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al.: Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025)
7. Hao, Y., Chen, Y., Zhang, Y., Fan, C.: Large language models can plan your travels rigorously with formal verification tools. *arXiv preprint arXiv:2404.11891* (2024)
8. Horvitz, E.: Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712* (2023)
9. Hu, M., Zhao, P., Xu, C., Sun, Q., Lou, J., Lin, Q., Luo, P., Rajmohan, S., Zhang, D.: Agentgen: Enhancing planning abilities for large language model based agent via environment and task generation. *arXiv preprint arXiv:2408.00764* (2024)
10. Kanoje, S., Girase, S., Mukhopadhyay, D.: User profiling for recommendation system. *arXiv preprint arXiv:1503.06555* (2015)

11. Kong, Y., Ruan, J., Chen, Y., Zhang, B., Bao, T., Shi, S., Du, G., Hu, X., Mao, H., Li, Z., et al.: Tptu-v2: Boosting task planning and tool usage of large language model-based agents in real-world systems. arXiv preprint arXiv:2311.11315 (2023)
12. Li, Y., Wen, H., Wang, W., Li, X., Yuan, Y., Liu, G., Liu, J., Xu, W., Wang, X., Sun, Y., Kong, R.: Personal llm agents: Insights and survey about the capability, efficiency and security. arXiv preprint arXiv:2401.05459 (2024)
13. Liang, Y., Wu, C., Song, T., Wu, W., Xia, Y., Liu, Y., Ou, Y., Lu, S., Ji, L., Mao, S., et al.: Taskmatrix. ai: Completing tasks by connecting foundation models with millions of apis. *Intelligent Computing* **3**, 0063 (2024)
14. Liu, X., Ni, Z.: Role-playing prompt framework: Generation and evaluation. arXiv preprint arXiv:2406.00627 (2024)
15. Ravichandran, K., Gomasta, S.: Leveraging translation for optimal recall: Tailoring llm personalization with user profiles. arXiv preprint arXiv:2402.13500 (2024)
16. de la Rosa, T., Gopalakrishnan, S., Pozanco, A., Zeng, Z., Borrajo, D.: Tripal: Travel planning with guarantees by combining large language models and automated planners. arXiv preprint arXiv:2406.101 (2024)
17. Shakshuki, E., Koo, H.H., Benoit, D., Silver, D.: A distributed multi-agent meeting scheduler. *Journal of Computer and System Sciences* **74**(2), 279–296 (2008)
18. Song, Y., Xiong, W., Zhu, D., Wu, W., Qian, H., Song, M., Huang, H., Li, C., Wang, K., Yao, R., et al.: Restgpt: Connecting large language models with real-world restful apis. arXiv preprint arXiv:23006.06624 (2023)
19. Sreedhar, K., Chilton, L.: Simulating human strategic behavior: Comparing single and multi-agent llms. arXiv preprint arXiv:2402.08189 (2024)
20. Topsakal, O., Akinci, T.: Creating large language model applications utilizing langchain: A primer on developing llm apps fast. In: *International Conference on Applied Engineering and Natural Sciences*, vol. 1, pp. 1050–1056 (2023)
21. Valmeekam, K., Marquez, M., Sreedharan, S., Kambhampati, S.: On the planning abilities of large language models: A critical investigation. arXiv preprint arXiv:2305.15771 (2023)
22. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. *Advances in neural information processing systems* **30** (2017)
23. Vidivelli, S., Ramachandran, M., Dharunbalaji, A.: Efficiency-driven custom chatbot development: Unleashing langchain, rag, and performance-optimized llm fusion. *Computers, Materials & Continua* **80**(2) (2024)
24. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D.: Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* **35**, 24824–24837 (2022)
25. Wijerathne, O., Nimasha, A., Fernando, D., de Silva, N., Perera, S.: ScheduleMe: Multi-Agent Calendar Assistant. arXiv preprint arXiv:2509.25693 (2025)

26. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., Li, B., Jiang, L., Zhang, X., Wang, C.: Autogen: Enabling next-gen llm applications via multi-agent conversation framework. arXiv preprint arXiv:2308.08155 (2023)
27. Wu, S., Fung, M., Qian, C., Kim, J., Hakkani-Tur, D., Ji, H.: Aligning llms with individual preferences via interaction. arXiv preprint (2024)
28. Xie, C., Zou, D.: A human-like reasoning framework for multi-phases planning task with large language models. arXiv preprint arXiv:2405.18208 (2024)
29. Xie, J., Zhang, K., Chen, J., Zhu, T., Lou, R., Tian, Y., Xiao, Y., Su, Y.: Travelplanner: A benchmark for real-world planning with language agents. arXiv preprint arXiv:2402.01622 (2024)
30. Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T.L., Cao, Y., Narasimhan, K.: Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems* **36** (2024)
31. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: React: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629 (2022)
32. Zhang, S., Dinan, E., Urbanek, J., Szlam, A., Kiela, D., Weston, J.: Personalizing dialogue agents: I have a dog, do you have pets too? arXiv preprint arXiv:1801.07243 (2018)
33. Zheng, H., Mishra, S., Zhang, H., Chen, X., Chen, M., Nova, A., Hou, L., Cheng, H., Le, Q., Chi, E., et al.: Natural plan: Benchmarking llms on natural language planning. arXiv preprint arXiv:2406.04520 (2024)
34. Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., et al.: Judging llm-as-a-judge with mt-bench and chatbot arena. arXiv preprint arXiv:2306.05685 (2023)
35. Zhong, T., Liu, Z., Pan, Y., Zhang, Y., Zhou, Y., Liang, S., Wu, Z., Lyu, Y., Shu, P., Yu, X., et al.: Evaluation of openai o1: Opportunities and challenges of agi. arXiv preprint arXiv:2409.18486 (2024)
36. Zhong, W., Guo, L., Gao, Q., Ye, H., Wang, Y.: Memorybank: Enhancing large language models with long-term memory. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 19724–19731 (2024)

A Two plan details

- Sole Planning: This approach assumes that all necessary information for itinerary generation is available beforehand. The planner is solely responsible for constructing the itinerary using the provided details. The primary objective of this strategy is to assess the planner agent’s ability to generate optimal travel plans given a complete dataset.
- Two-Stage Planning: In this approach, itinerary generation is divided into two stages: data retrieval and planning. The first stage involves gathering relevant travel details, while the second stage focuses on generating the itinerary based on the retrieved information. This strategy is designed to evaluate the combined performance of both the data retrieval agent and the itinerary planner.

B Performance Metrics

We adopt the evaluation protocol of the TravelPlanner benchmark [29] and report five metrics: *Delivery Rate*, *Commonsense Constraint Pass Rate* (micro and macro), *Hard Constraint Pass Rate* (micro and macro), and *Final Pass Rate*. Below we provide precise definitions, with concrete examples for transparency.

Delivery Rate. The percentage of runs that successfully produce a schema-valid itinerary within the step budget (30 steps in our setting). A run fails if it exceeds the step limit, falls into a dead loop, or generates an invalid schema.

Commonsense Constraint Pass Rate. This metric evaluates whether itineraries respect implicit feasibility rules, even when they are not explicitly stated in the user query. Following [29], we evaluate eight commonsense dimensions: (i) *Within Sandbox* (the plan does not hallucinate entities outside the environment), (ii) *Complete Information* (all key components such as accommodation and transportation are specified), (iii) *Within Current City* (daily activities occur in the correct city), (iv) *Reasonable City Route* (city transitions follow a logical sequence), (v) *Diverse Restaurants* (restaurant choices are not repeated), (vi) *Diverse Attractions* (attractions are not repeated), (vii) *Non-conflicting Transportation* (no contradictory modes such as “flight” and “self-driving” in the same trip), and (viii) *Minimum Nights Stay* (accommodations respect stay requirements).

Hard Constraint Pass Rate. This metric measures adherence to user-specified requirements, such as budget, accommodation rules (e.g., no pets or smoking), accommodation type (entire, private, or shared), cuisine preferences, and transportation constraints (e.g., “no flights”).

Micro vs. Macro pass rates. Let P denote the set of all evaluated plans and C_p the set of constraints applicable to plan $p \in P$. Define $\mathbf{1}_{\text{passed}}(c, p) \in \{0, 1\}$ as an indicator that plan p satisfies constraint c , and $\mathbf{1}_{\text{passed}}(C_p, p) = 1$ if *all* constraints in C_p are satisfied (else 0). Then:

$$\text{Micro Pass Rate} = \frac{\sum_{p \in P} \sum_{c \in C_p} \mathbf{1}_{\text{passed}}(c, p)}{\sum_{p \in P} |C_p|}.$$

$$\text{Macro Pass Rate} = \frac{\sum_{p \in P} \mathbf{1}_{\text{passed}}(C_p, p)}{|P|}.$$

Intuitively, the micro score measures success *constraint by constraint* across all plans, while the macro score is stricter, counting a plan only if it satisfies *all* of its applicable constraints.

Final Pass Rate. The proportion of plans that simultaneously (i) are delivered, (ii) pass all applicable commonsense constraints, and (iii) pass all applicable hard constraints.

C Assessment Criteria

Each itinerary was evaluated using a 1-to-5 rating scale based on how well it aligned with the user profile and query. The criteria for assigning ratings were:

1. Lowest Personalization: The plan is generic or completely misaligned with the user’s profile and query. No effort is evident in catering to user preferences or constraints.
2. Poor Personalization: Some elements loosely align with the user’s profile, but most aspects are irrelevant or mismatched. Minimal effort was made to personalize the itinerary.
3. Moderate Personalization: The plan partially addresses user preferences and constraints. It includes both relevant and irrelevant elements, showing some personalization but lacking depth.
4. Good Personalization: The plan mostly aligns with the user profile and query. It includes tailored elements, though minor gaps or missed opportunities exist.
5. Excellent Personalization: The plan is highly tailored to the user’s profile and query. There are no significant misalignments, and the plan demonstrates a deep understanding of user preferences and constraints with highly relevant details.

D Reproducibility Bundle

Environment and Performance. Experiments ran on Windows 10 (64-bit, build 10.0.26100) with Python 3.11 on x64 CPUs. Planning proceeded for up to five iterations with a 300s timeout and default temperature 0.7 (seeds not fixed). Measured end-to-end latency ranged from 30–300s depending on itinerary complexity, with typical sessions involving 1–3 LLM calls per agent, 5–15 database queries, and 1.5k–7k tokens (0.5k–2k input; 1k–5k output), at an estimated cost of USD \$0.00015–\$0.0007 (GPT-4o). While introducing communication/memory overhead compared to single-agent baselines, the multi-agent design enabled modular specialization, parallelism, and feedback-driven refinement.

E Prompts Used in Evaluation

In this section, we provide the exact prompts used for different components of our evaluation. Each prompt is shown in full for reproducibility.

E.1 LLM-as-a-Judge Prompt

We employed GPT-4o as an automatic evaluator (“LLM-as-a-Judge”) to assess the personalization quality of generated travel plans. The following prompt was used:

Judge Prompt

Context:

You are evaluating the personalization quality of a travel plan based solely on the user profile and query. Personalization refers to how well the plan aligns with the user's stated preferences, constraints, and unique details. Do not consider travel dates, budget constraints, or the validity of travel logistics (e.g., transportation, origin, destination). These factors are assessed separately. Your focus should be exclusively on how well the plan matches the user profile and stated interests.

Instructions:

You will be provided with the following:

- A User Profile detailing preferences, constraints, and travel interests.
- A User Query specifying the travel requirements.
- A Travel Plan generated for the user.

Rate the personalization of the travel plan on a scale from 1 to 5, following the guidelines below.

Output strictly in the format: Rating: X *** Explanation: Y

Personalization Rating Scale (1 to 5):

1. Lowest: Generic or misaligned with the user profile and query.
2. Poor: Some loose alignment, mostly irrelevant or mismatched.
3. Moderate: Partial alignment, includes both relevant and irrelevant elements.
4. Good: Mostly aligns with profile and query, with minor gaps.
5. Excellent: Highly tailored to the profile and query, deeply relevant and consistent.

Your Task:

Assign a rating (1--5) and provide a short explanation (2--3 sentences) highlighting which aspects of the plan matched or missed the user profile and query.

User Query:

"Please plan a trip for me starting from Sarasota to Chicago for 3 days, from March 22nd to March 24th, 2022. The budget for this trip is set at \$1,900."

User Profile:

"She is a 25-year-old data analyst who loves painting. She is single and lives with her best friend. She holds a degree in statistics. She enjoys attending art galleries and has a small

studio in her apartment where she paints. She also enjoys cooking Italian dishes."

Travel Plan:

"Travel Plan:\n\nDay 1: March 22, 2023\nCurrent City: from Sarasota to Chicago\nTransportation: Flight Number: F0123456, from Sarasota to Chicago, Departure Time: 09:00, Arrival Time: 11:30"

Example Output Format:

Rating: 5 *** Explanation: "The travel plan is highly personalized to the user's profile, focusing on her love for art and Italian cuisine. It includes visits to art galleries and museums, such as The Art Institute of Chicago and the Chicago Cultural Center, which align with her interests in painting. The dining options predominantly feature Italian cuisine, catering to her culinary preferences. The accommodation is conveniently located near art galleries, further enhancing the tailored experience."

E.2 Chatbot Orchestrator Prompt

This prompt serves as the central supervisor for the travel planning system. It instructs the chatbot on how to collect user details, route queries to the appropriate agents (calendar, data retrieval, itinerary generation, feedback, and suggestion), and return responses in a structured JSON format. The orchestrator ensures that the workflow remains consistent, personalized, and user-friendly.

Central supervisor Prompt

You are a dedicated travel planning chatbot designed to create personalized and well-organized trips.

Your Responsibilities:

1. Greeting \& Introduction: Start by greeting the user and explaining that you're here to help plan their trip. Inform them that currently you can assist them with:
 - Calendar checks
 - Add events to user's Google calendar
 - Data retrieval for travel details (e.g., restaurants, flights, attractions)
 - Itinerary planning
 - Suggestions for attractions, hotels, restaurants in specific areas

2. Query Construction : Before moving forward, gather essential details for building a travel query. You will ask the user for :
 - Departure location(required)
 - Destination(required)
 - Budget(optional)
 - Preferences (optional: activities, accommodations, food, etc.)
3. Information Collection:

Inquire about specific details that are required for each agent as follows:

 - **calendar_agent**: User's travel dates and any potential conflicts with Google Calendar events.
 - **data_retrieval_agent**: Budget and activity preferences to gather relevant data for the trip (e.g., flights, restaurants, local attractions).
 - **itinerary_agent**: Once data is gathered, this agent will create a personalized itinerary for the user.
4. Agent Routing: Based on the collected information, determine which agent to route the user to:
 - **calendar_agent**: Check for any calendar conflicts with the user's travel dates and add calendar events to the google calendar.
 - **data_retrieval_agent**: Fetch the relevant travel data after the user provides their preferences and budget.
 - **itinerary_agent**: Generate the itinerary after gathering travel data.
 - **feedback_agent**: Handle user feedback and make specific changes to existing travel plans. This agent can understand what changes are needed and fetch additional data if required.
 - **suggestion_agent**: Provide recommendations for attractions, hotels, restaurants, etc. in specific areas without creating full travel plans.
 - **human_interrupt**: Allow the user to interact directly and make any changes to their itinerary or provide additional information.
5. Response Handling:
 - **Structured Output**: Ensure all responses are in JSON format with the following keys:
 - 'next': The next agent to route to ('calendar_agent', 'data_retrieval_agent', 'itinerary_agent', 'feedback_agent', 'suggestion_agent', 'human_interrupt', or 'FINISH').
 - 'messages': The message content to send to the user.

6. Information Validation: If any required information is missing or incomplete, gather more information from the user.
7. Finalization: Once the itinerary is complete, ask the user if they would like to add anything else to the plan or finalize the itinerary.

Communication Style:

- Use a friendly, clear, and informative tone.
- Ensure all questions are concise and easy to understand.
- When you are done with the itinerary, always route to the 'human_interrupt' agent for review or additional input.

Example Workflow:

1. ****Chatbot****: "Hello! I'm here to help you plan your perfect trip . Could you please share your preferred destinations and travel dates?"
2. ****User****: "I'm traveling to New Jersey from March 10 to March 17."
3. ****Chatbot****: "Noted! Now, could you share your budget and preferences for activities, restaurants, or accommodations?"
4. ****User****: [Provides details]
5. ****Chatbot****: "Let me check your calendar for any conflicts with these dates..."
6. ****calendar_agent****: [Checks user's Google Calendar for conflicts]
7. ****Chatbot****: "I see that you have a meeting on March 12 at 3 PM. Would you like to adjust your travel dates or keep this event?"
8. ****User****: "I'll keep it, but please add a reminder."
9. ****Chatbot****: [Routes to 'data_retrieval_agent' to fetch relevant data]
10. ****data_retrieval_agent****: [Fetches data: restaurants, flight information, and local attractions in New Jersey]
11. ****Chatbot****: "I've gathered all the details! Now I'm creating your personalized itinerary. Just a moment."
12. ****itinerary_agent****: [Creates itinerary]
13. ****Chatbot****: "Here's your personalized itinerary! Would you like to add anything else?"
14. ****User****: "No, that's all. Thank you!"
15. ****Chatbot****: "Would you like me to add this event to your Google Calendar?"
16. ****User****: "Yes, please."
17. ****Chatbot****: "Let me add this event your calendar.Event details : 2nd March - 5th March Trip to Miami from NewYork."
18. ****calendar_agent****: [Adds events to Google Calendar]
19. ****Chatbot****: "FINISH"